L3-INFO Algorithmique

Quick 1 – 3 octobre 2025 – Durée 1 h

Une feuille A4 manuscrite recto-verso autorisée, ainsi que les dictionnaires bilingues.
Les dispositifs électroniques sont interdits, à l'exception des montres non connectées.
Le barème est indicatif.

Vous êtes vivement encouragés à illustrer vos recherches et vos réponses par des schémas.

---

1 Structure de données (7 points)

On s'intéresse dans cet exercice à la structure de données *Bloc* décrite par le type abstrait ci-dessous.

Initialiser(B)
{ **Donnée-résultat** *: un Bloc B*
Post-condition *: B est initialisé avec un contenu vide*
Effet de bord *: B est modifié }*

EstVide(B)
{ **Donnée** *: un Bloc B*
Résultat *: un booléen vrai ssi B est vide }*

Déposer(B,x)
{ **Donnée-résultat** *: un Bloc B*
Donnée *: un élément x*
Post-condition *: x est ajouté dans B*
Effets de bord *: B est modifié }*

Extraire(B)
{ **Donnée-résultat** *: un Bloc B*
Résultat *: l'élément x le plus récent de B*
Pré-condition *: B est non vide*
Post-condition *: supprime l'élément x le plus récent de B*
Effets de bord *: B est modifié }*

Effacer(B,x)
{ **Donnée-résultat** *: un Bloc B*
Donnée *: un élément x*
Post-condition *: supprime l'élément de B le plus récent dont la valeur est égale à x*
Effets de bord *: B est modifié, sauf si aucun élément de B n'est égal à x }*

(1 pt) a) Illustrez le fonctionnement de cette structure en choisissant une suite d'opérations qui montre tout ce qui peut se passer, et en décrivant le contenu du *Bloc* par un schéma après chaque étape.

b) Proposez une implémentation de ce type, autrement dit :

(2 pts) (i) Décrivez une représentation concrète des objets de type *Bloc*, à l'aide d'autres types de données classiques (entiers, booléens, tableaux, listes chaînées).
Vous accompagnerez vos explications de schémas.
Note : Si vous utilisez un tableau, vous ne tiendrez pas compte des cas où on essaye de stocker plus d'éléments que le tableau ne le permet.

(3 pts) (ii) Écrivez un ensemble de fonctions qui réalisent les opérations spécifiées ci-dessus.

(1 pt) (iii) Donnez (**sans justification**) le coût de chacune de vos opérations.

Solution:

a) Il s'agit d'une pile classique avec une opération supplémentaire $Efface$.
Attention à bien lire la spécification :
— x peut être absent du bloc ; si c'est le cas, alors $Effacer$ ne la modifie pas.
— Si x est présent en plusieurs exemplaires, alors c'est celui le plus « haut » dans la pile qui est effacé (et seulement lui).
— L'ordre des autres éléments n'est pas modifié.

b) Le schéma n'est pas qu'une question de cours sur les piles, il doit expliciter comment **votre** représentation permet de coder toutes les opérations, comment elle garde trace de l'état courant de la pile, etc.

Plusieurs possibilités de représentations :
— Utiliser une pile et programmer Effacer récursivement (on détourne alors la pile de récursivité pour « mettre de côté » les éléments situés au-dessus de x).
— Utiliser un tableau + un nombre d'éléments, et décaler les éléments à droite de x.
— Utiliser un tableau + un nombre d'éléments, et marquer les cases « mortes ».
— Utiliser une liste chaînée et supprimer le maillon de x en place.
— ...
Réorganiser la pile après suppression de x peut être plus ou moins coûteux. Mais dans tous les cas, avant de l'effacer, il faut quand même le trouver, ce qui est linéaire en la taille courante de la pile.

2 Écriture d'algorithme (6 points)

L'objectif de cet exercice est l'écriture et l'analyse d'algorithmes permettant de déterminer l'élément apparaissant le plus de fois dans un tableau donné.

Par exemple, dans $T = [E, B, E, E, I, C, B, C]$ l'élément E apparaît 3 fois alors que les autres n'apparaissent que 1 ou 2 fois, c'est donc E la valeur cherchée.

En cas d'égalité, parmi les « premiers *ex æquo* », on choisira de renvoyer celui qui apparaît le plus à gauche dans T. Par exemple pour $[R, E, B, E, I, C, B, C]$, les valeurs B, C et E apparaissent 2 fois chacune, on renvoie E car parmi les plus fréquents c'est lui qui est situé le plus à gauche dans T.

a) Algorithme naïf

(2 pts) (i) Écrivez un algorithme naïf qui résout ce problème.

(1 pt) (ii) Déterminez et justifiez le coût au pire de votre algorithme.

Solution:

(i)

```
MAJO_NAIF(T)
    max := 0
    pour i := 0 à n − 1 faire
        occurrences := 1
        pour j := i + 1 à n − 1 faire
            si T[j] = T[i] alors
                occurrences := occurrences + 1
            si occurrences > max alors
                max := occurrences
                val := T[i]
    renvoyer val
```

(ii) On s'intéresse aux comparaisons entre éléments de T.

Il est facile de voir que l'algorithme effectue exactement $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$ comparaisons quel que soit le contenu du tableau, il est donc quadratique.

b) On admet pour cette partie disposer d'un algorithme qui trie un tableau de taille n en effectuant au pire $n \log n$ comparaisons.

On propose alors l'algorithme suivant :

```
MAJORITAIRE_AVEC_TRI(T)
    Trier(T)
    max := 0
    i := 0
    tant que i < n faire
        candidat := T[i]
        occurrences := 0
        tant que i < n  et  T[i] = candidat faire
            occurrences := occurrences + 1
            i := i + 1
        si occurrences > max alors
            max := occurrences
            valeur := candidat
    renvoyer valeur
```

(2 pts) (i) Quel est le coût total de cet algorithme ?
Attention à bien détailler vos calculs et vos justifications.

(1 pt) (ii) Dans certains cas, cet algorithme ne répond pas au problème posé.
Lesquels et pourquoi ?

Solution:

(i) Le tri préalable demande $\mathcal{O}(n \log n)$ comparaisons.

Attention, ensuite, même si on a deux boucles imbriquées, il serait inexact de conclure à un coût quadratique.

En effet, i n'est incrémenté que dans la boucle interne et jamais réinitialisé, il ne peut donc augmenter que n fois.

À chaque fois qu'on effectue une comparaison, on incrémente i, sauf la comparaison qui fait sortir de la boucle interne.

Par ailleurs, on entre dans la boucle interne au moins une fois par tour de boucle externe (car *candidat* est initialisé à $T[i]$), il n'y a donc pas plus de n tours de boucle externe, c'est-à-dire pas plus de n comparaisons non comptabilisées par i.

Au pire (tableau d'éléments tous distincts) on effectuera donc $2n$ comparaisons (de chaque $T[i]$ avec lui-même et avec $T[i+1]$).

On est donc en temps $n \log n + 2n = \mathcal{O}(n \log n)$.

(ii) Comme cet algorithme réordonne le tableau, on perd l'information de l'élément le plus à gauche. En cas d'ex aequo, il peut donc renvoyer une autre valeur que celle voulue.

3 Analyse de coût (7 points)

Les deux algorithmes étudiés dans cet exercice prennent chacun en entrée un tableau T de n entiers dont les valeurs sont toutes comprises entre 0 et $n-1$.

Précisons qu'il est possible que deux cellules distinctes de T possèdent des valeurs identiques.

Dans tout l'exercice, les mesures de coûts sont le nombre exact d'appels à la fonction Traiter, et **uniquement** ces appels.

a) On s'intéresse d'abord à l'algorithme :

```
ALGO_EGAL(T)
pour i := 0 à n-1 faire
    si T[i]=i alors
        Traiter T[i]
```

(1 pt) (i) Quel est son coût au pire ?
Justifiez et exhibez un cas défavorable.

(1 pt) (ii) Quel est son coût au mieux ?
Justifiez et exhibez un cas favorable.

(1½ pts) (iii) Quel est son coût en moyenne ?
On suppose que chaque cellule de T contient un entier choisi au hasard uniformément entre 0 et $n-1$. La valeur de chaque cellule est indépendante des autres.

b) Répondez ensuite aux mêmes questions pour ce second algorithme :

```
ALGO_INFERIEUR(T)
pour i := 0 à n-1 faire
    si T[i] ⩽ i alors
        Traiter T[i]
```

(1 pt) (i) Quel est son coût au pire ?
Justifiez et exhibez un cas défavorable.

(1 pt) (ii) Quel est son coût au mieux ?
Justifiez et exhibez un cas favorable.

(1½ pts) (iii) Quel est son coût en moyenne ?
On suppose que chaque cellule de T contient un entier choisi au hasard uniformément entre 0 et $n-1$. La valeur de chaque cellule est indépendante des autres.

Solution:

a) (i) $C_{min}(n) = 0$ (par exemple si $T[i] = 0$ pour tout i, sauf $T[0] = 1$) et on ne peut pas faire moins que 0 !

(ii) $C_{max}(n) = n$ (si $T[i] = i$ pour tout i) et on ne peut pas faire plus de n puisqu'on est dans une boucle Pour avec un seul Traiter par itération.

(iii) Avec cette hypothèse, chaque test a une probabilité $\frac{1}{n}$ d'être vrai et ils sont indépendants. D'où $C_{moy}(n) = n \times \frac{1}{n} = 1$.

b) (i) $C_{min}(n) = 1$ (par exemple si $T[i] = n-1$ pour tout i) et on ne peut pas faire moins car $T[n-1] \leqslant n-1$ est toujours vrai.

(ii) $C_{max}(n) = n$ (par exemple si $T[i] = i$ pour tout i) et on ne peut pas faire plus de n pour les mêmes raisons que l'algo précédent.

(iii) Avec cette hypothèse, le test $T[i] \leqslant i$ a une probabilité $\frac{i+1}{n}$ d'être vrai et ils sont indépendants. D'où $C_{moy}(n) = \sum_{1}^{n} \frac{i}{n} = \frac{1}{n} \times \frac{n(n+1)}{2} = \frac{n+1}{2}$.